

## The Definitive Guide to Power Query M

MODULE

10

# Performance Optimisation and Advanced Archi

Master VertiPaq compression, column cardinality, star schema construction in M, Table.Buffer strategy, refresh architecture, Incremental Refresh, Composite Models, large-model storage, and every M-level technique that keeps Power BI models fast, lean, and scalable in production.

# Module 10: Performance Optimisation and Advanced Architecture

---

## Module Contents

### ■ Module Overview

#### ■ Lesson 1: How Power BI Stores Data — VertiPaq Internals

- Column-Store Compression and Dictionary Encoding
- Run-Length Encoding and Value Encoding
- Cardinality and Its Impact on Model Size
- Measuring Model Size with DAX Studio

#### ■ Lesson 2: Star Schema Construction in M

- Why Star Schema Matters for Performance
- Building DimDate in Pure M
- Building Dimension Tables from Fact Data
- Surrogate Keys — When and How
- Role-Playing Dimensions

#### ■ Lesson 3: Reducing Cardinality and Model Size

- Removing High-Cardinality Columns
- Rounding and Binning Numeric Columns
- Date and DateTime — The Biggest Cardinality Mistake
- Text Column Optimisation
- Integer Keys vs Text Keys

#### ■ Lesson 4: Table.Buffer — Strategic Caching in M

- When Table.Buffer Helps and When It Hurts
- Multi-Reference Queries and Double Evaluation
- Buffering Lookup Tables
- List.Buffer for List-Level Caching

#### ■ Lesson 5: Refresh Architecture and Scheduling

- Full Refresh vs Incremental Refresh
- Incremental Refresh Configuration in M
- Hybrid Tables — Real-Time Partition
- Parallel Refresh and Dataset Partitioning
- Power BI Premium vs Shared Capacity Refresh Limits

#### ■ Lesson 6: Large Models, Composite Models, and DirectQuery

- Large Model Storage Format
- Composite Models — Import + DirectQuery
- Dual Storage Mode for Dimensions
- Aggregation Tables
- DirectQuery Performance Optimisation in M

#### ■ Lesson 7: Advanced M Performance Patterns

- Pattern 1: Lazy Loading with Function Queries
- Pattern 2: Parallelising Independent Queries
- Pattern 3: Diagnostic Watermark Query
- Pattern 4: Column Pruning Pipeline

- Pattern 5: Refresh-Time Config from SharePoint

### ■ **Demonstration 1: Diagnosing Model Bloat with DAX Studio**

### ■ **Demonstration 2: Building a Star Schema from a Flat File**

### ■ **Lab 10: Performance Optimisation and Advanced Architecture**

- Exercise 1: VertiPaq Audit and Cardinality Reduction
- Exercise 2: Star Schema Construction in M
- Exercise 3: Incremental Refresh and Table.Buffer
- Exercise 4: Composite Model and Aggregation Table

### ■ **Module Review and Takeaways**

## Module Overview

Performance in Power BI is determined by decisions made at every layer of the stack: how the M query loads and shapes data, how columns are typed and sized, how the model is structured, and how refreshes are scheduled. A technically correct model that loads a 20-million-row flat file with 60 text columns into a single table will be slow, large, and difficult to maintain — regardless of how well the DAX is written. This module addresses every M-level and architecture-level lever available to the Power BI developer.

The module begins with VertiPaq internals — the in-memory column-store engine that powers Power BI Import mode — because understanding compression mechanics directly determines which M transformations improve performance and which make it worse. It then covers star schema construction, cardinality reduction, Table.Buffer strategy, refresh architecture, and the advanced storage options available in Power BI Premium.

### NOTE

This module builds on Module 9 (Query Folding). The folding patterns covered there are prerequisites — especially query folding for Incremental Refresh. Lessons 5 and 6 also reference DAX concepts (measures, relationships) at a high level. Full DAX coverage is in the companion DAX course.

## Before You Begin

- Install DAX Studio (free, open-source) — essential for VertiPaq analysis.
- Install Tabular Editor 2 (free) for model metadata inspection.
- Have a Power BI model with at least one large fact table (500K+ rows) for exercises.
- Ensure Power BI Desktop is version October 2023 or later for Hybrid Table support.
- Complete Modules 1–9, particularly Module 9 (Query Folding).

## LESSON 1

## How Power BI Stores Data — VertiPaq Internals

After completing this lesson, you will be able to:

- Explain how VertiPaq compresses column data using dictionary and run-length encoding.
- Define cardinality and explain its direct relationship with model size and query speed.
- Measure column-level size and cardinality using DAX Studio's VertiPaq Analyzer.
- Identify which columns consume the most model memory.

### Column-Store Architecture

VertiPaq stores each column independently as a compressed array — this is called a *column-store* or *columnar* format. When Power BI evaluates a DAX expression that touches three columns from a 50-million-row table, it only reads those three column arrays — not all 60 columns. This is why column selection and pruning in M has a direct and measurable impact on model performance.

VertiPaq applies two compression algorithms in sequence: **Value Encoding** (integer arithmetic to reduce bit depth) and **Run-Length Encoding (RLE)** (consecutive repeated values stored as a single value + count pair). Compression effectiveness depends almost entirely on **cardinality** — the number of distinct values in a column:

| Distinct Values       | Compression | Typical Ratio | Explanation                                             |
|-----------------------|-------------|---------------|---------------------------------------------------------|
| 100 (e.g. Country)    | Excellent   | ~99.9%        | Dictionary: 100 entries; entire column is integers 0–99 |
| 10,000 (e.g. City)    | Good        | ~95%          | Dictionary: 10K entries; reasonable compression         |
| 1,000,000 (e.g. Name) | Poor        | ~40%          | Large dictionary; compression limited                   |
| Unique (e.g. OrderID) | None        | <5%           | Every value distinct; dictionary IS the column data     |
| DateTime (microsec)   | None        | <5%           | Each timestamp likely unique — never store as datetime  |

**WARNING**

A DateTime column (e.g. 2024-03-15 14:32:07.123) has near-unique values — each row is a different timestamp. VertiPaq cannot compress it. A 10-million-row fact table with a DateTime column can be 300 MB for that column alone. Always split DateTime into a Date column (joins to DimDate) and a TimeOfDay column if time granularity is required. Date alone has at most 365 distinct values per year.

### Measuring Model Size with DAX Studio

```
-- DAX Studio: Server Timings + VertiPaq Analyzer
-- 1. Open DAX Studio → Connect to Power BI Desktop model
-- 2. Advanced → View Metrics → VertiPaq Analyzer
-- 3. Columns tab: sort by Total Size (MB) descending

-- Key metrics to check per column:
-- Total Size (MB) : compressed storage on disk / in RAM
-- Dictionary Size : size of the distinct-value lookup table
-- Data Size : compressed column data (value/RLE encoded)
-- Cardinality : number of distinct values
-- % DB Size : fraction of total model RAM

-- Red flags in VertiPaq Analyzer:
```

```
-- Cardinality = RowCount → fully unique column (ID, timestamp, free-text)
-- Dictionary Size > Data Size → very high cardinality; compression failing
-- Single column > 20% DB Size → dominant bloat source; investigate and reduce

-- Example output for a problematic model:
-- Column | Cardinality | Total Size
-- FactSales[CreatedAt] | 9,847,234 | 187 MB ← DateTime: near-unique
-- FactSales[Notes] | 4,209,881 | 143 MB ← Free-text: high cardinality
-- FactSales[TransactionRef] | 9,847,234 | 312 MB ← Unique string ID
-- FactSales[SaleDate] | 1,827 | 4 MB ← Date only: low cardinality ✓
```

## LESSON 2

## Star Schema Construction in M

**After completing this lesson, you will be able to:**

- Explain why a star schema outperforms a flat denormalised table in Power BI.
- Build a complete DimDate table in pure M without any external data source.
- Extract dimension tables from fact data using Table.Distinct and Table.SelectColumns.
- Implement surrogate integer keys for dimension-fact relationships.

### Why Star Schema Outperforms Flat Tables

A flat denormalised table repeats dimension attributes on every fact row. If 10 million orders each carry the customer's Name, City, Country, and Segment as text columns, VertiPaq stores 10 million copies of those strings. In a star schema, the fact table stores only a CustomerID integer key; the DimCustomer table stores each customer's attributes once. VertiPaq compresses the integer key column to near-nothing (low cardinality), and DimCustomer has only one row per customer.

| Flat (Denormalised) Table                  | Star Schema                                   |
|--------------------------------------------|-----------------------------------------------|
| Fact table = 10M rows × 40 columns         | Fact table = 10M rows × 6 integer keys        |
| Text attributes repeated 10M times         | Text attributes stored once per dimension row |
| High cardinality on description columns    | Dimension tables: 100–100K rows only          |
| DAX relationships not possible             | DAX relationships on integer keys             |
| Cross-filter between dimensions impossible | Slicers cross-filter across all dimensions    |
| PBIX file: 800 MB+                         | PBIX file: 50–100 MB                          |

### Building DimDate in Pure M

```
// Complete DimDate table – no external data source required
// Generates one row per calendar day between StartDate and EndDate

let
    StartDate = #date(2018, 1, 1),
    EndDate = #date(2030, 12, 31),
    DayCount = Duration.Days(EndDate - StartDate) + 1,

    // Generate list of every date in the range
    DateList = List.Dates(StartDate, DayCount, #duration(1,0,0,0)),

    // Build table from the list
    DateTable = Table.FromList(DateList, Splitter.SplitByNothing(), {"Date"}),
    Typed = Table.TransformColumnTypes(DateTable, {"Date", type date}),

    // Add all calendar attributes
    WithYear = Table.AddColumn(Typed, "Year", each Date.Year([Date]), Int64.Type),
    WithQtr = Table.AddColumn(WithYear, "Quarter", each Date.QuarterOfYear([Date]), Int64.Type),
    WithQtrLabel = Table.AddColumn(WithQtr, "QuarterLabel",
        each "Q" & Text.From(Date.QuarterOfYear([Date]))
        & " " & Text.From(Date.Year([Date])), type text),
    WithMonth = Table.AddColumn(WithQtrLabel, "MonthNum", each Date.Month([Date]), Int64.Type),
    WithMonthName = Table.AddColumn(WithMonth, "MonthName", each Date.MonthName([Date]), type text),
```

```

WithMonthShort = Table.AddColumn(WithMonthName, "MonthShort", each Date.ToText([Date], "MMM"),
type text),
WithWeekNum = Table.AddColumn(WithMonthShort, "WeekNumber", each Date.WeekOfYear([Date]),
Int64.Type),
WithDOW = Table.AddColumn(WithWeekNum, "DayOfWeek", each Date.DayOfWeek([Date]), Int64.Type),
WithDOWName = Table.AddColumn(WithDOW, "DayName", each Date.DayOfWeekName([Date]), type text),
WithDayNum = Table.AddColumn(WithDOWName, "DayOfMonth", each Date.Day([Date]), Int64.Type),
WithIsWeekend = Table.AddColumn(WithDayNum, "IsWeekend",
each Date.DayOfWeek([Date]) >= 5, type logical),
WithYearMonth = Table.AddColumn(WithIsWeekend, "YearMonthKey",
each Date.Year([Date])*100 + Date.Month([Date]), Int64.Type),
WithDateKey = Table.AddColumn(WithYearMonth, "DateKey",
each Date.Year([Date])*10000
+ Date.Month([Date])*100
+ Date.Day([Date]), Int64.Type),

// Final column selection and ordering
Result = Table.SelectColumns(WithDateKey,
{"DateKey", "Date", "Year", "Quarter", "QuarterLabel", "MonthNum", "MonthName",
"MonthShort", "WeekNumber", "DayOfWeek", "DayName", "DayOfMonth", "IsWeekend", "YearMonthKey"})
in
Result

```

## Building Dimension Tables from Fact Data

```

// Extract DimProduct from FactSales – deduplicate and assign surrogate key
let
// Start from the fact table
Fact = FactSales,

// Select product attribute columns and deduplicate
ProductCols = Table.SelectColumns(Fact,
{"ProductCode", "ProductName", "Category", "SubCategory", "Brand"}),
Distinct = Table.Distinct(ProductCols, "ProductCode"), // unique on natural key

// Sort for consistent surrogate key assignment
Sorted = Table.Sort(Distinct, {"ProductCode", Order.Ascending}),

// Add integer surrogate key – starts at 1
WithKey = Table.AddIndexColumn(Sorted, "ProductKey", 1, 1, Int64.Type),

// Reorder: key first, then natural key, then attributes
Result = Table.ReorderColumns(WithKey,
{"ProductKey", "ProductCode", "ProductName", "Category", "SubCategory", "Brand"}),

Typed = Table.TransformColumnTypes(Result,
{{"ProductKey", Int64.Type}, {"ProductCode", type text},
{"ProductName", type text}, {"Category", type text},
{"SubCategory", type text}, {"Brand", type text}})
in
Typed

// Join surrogate key back to FactSales
// Add ProductKey to FactSales via a lookup join:
let
DimProd = DimProduct, // already created above
KeyMap = Table.SelectColumns(DimProd, {"ProductCode", "ProductKey"}),
Joined = Table.NestedJoin(FactSales, "ProductCode",

```

```
KeyMap,"ProductCode","_k",JoinKind.LeftOuter),
Expanded = Table.ExpandTableColumn(Joined,"_k",{"ProductKey"},"ProductKey"),
// Now remove the original text ProductCode from the fact table
Result = Table.RemoveColumns(Expanded,
{"ProductCode","ProductName","Category","SubCategory","Brand"})
in
Result
```

## LESSON 3

## Reducing Cardinality and Model Size

## After completing this lesson, you will be able to:

- Identify and remove high-cardinality columns that add size without analytical value.
- Replace DateTime columns with Date and separate Time dimension references.
- Round and bin numeric measures to reduce distinct value counts.
- Replace high-cardinality text keys with integer surrogate keys.

## The DateTime Anti-Pattern

The single most common model bloat cause is loading DateTime columns as type `datetime`. A datetime value has microsecond precision — in a table with 10 million unique transactions, every timestamp is different. VertiPaq cannot compress a column where every value is distinct. The solution is always the same: split datetime into date and time.

```
// ■ ANTI-PATTERN: datetime column on a fact table – ~200 MB per 10M rows
Table.TransformColumnTypes(Source, {"CreatedAt", type datetime}))

// ✓ CORRECT: split into date (joins to DimDate) and time bucket (optional)
Table.AddColumn(Source, "SaleDate",
each Date.From([CreatedAt]), type date), // low cardinality ✓
Table.AddColumn(WithDate, "HourOfDay",
each Time.Hour(DateTime.Time([CreatedAt])), Int64.Type), // 0-23: 24 distinct values ✓

// Remove the original datetime after splitting
Table.RemoveColumns(WithTime, {"CreatedAt"})

// Result: SaleDate column → ~1,800 distinct values (5 years of data)
// Instead of: CreatedAt → 10,000,000 distinct values (near-unique)
// RAM saved: ~180 MB for this one change
```

## Rounding and Binning Numeric Columns

```
// High-precision decimal columns have high cardinality – round to meaningful precision
// Revenue with 6 decimal places: e.g. 1234.567891 – nearly unique
// Revenue rounded to 2 decimal places: e.g. 1234.57 – dramatically fewer distinct values

// Round to 2 decimal places in M (do this BEFORE loading to model)
Table.TransformColumns(Source, {
{"Revenue", each Number.Round(_, 2), type number},
{"UnitPrice", each Number.Round(_, 2), type number},
{"CostBasis", each Number.Round(_, 4), type number} // keep 4dp for margin calcs
})

// Binning – create a RangeBand column for bucket analysis instead of raw decimals
Table.AddColumn(Source, "RevenueBand",
each
if [Revenue] < 100 then "0-99"
else if [Revenue] < 500 then "100-499"
else if [Revenue] < 1000 then "500-999"
else if [Revenue] < 5000 then "1K-5K"
else "5K+"
, type text)
// RevenueBand: 5 distinct values → near-perfect VertiPaq compression
```

```
// Use RevenueBand for slicers/charts; keep Revenue for SUM measures
```

## Columns to Remove Before Loading

Many source systems export columns that are internally useful but add no analytical value to the Power BI model. These should be removed in M — not in DAX — because columns removed in M are never loaded into VertiPaq at all:

| Column Type                    | Problem                           | Action                                          |
|--------------------------------|-----------------------------------|-------------------------------------------------|
| Free-text notes / comments     | Near-unique cardinality           | Remove unless full-text search needed           |
| Internal GUID / UUID fields    | Fully unique; 0% compression      | Replace with integer surrogate key              |
| Audit columns (ModifiedBy etc) | High cardinality, rarely queried  | Remove unless required by governance            |
| Duplicate / redundant keys     | Duplicate storage                 | Keep only the joining key                       |
| Intermediate computed columns  | Replaced by DAX measures          | Never load calc-ready columns — use DAX         |
| Untyped 'any' columns          | Prevent VertiPaq optimisation     | Always type all columns in TransformColumnTypes |
| Binary / image data            | Enormous; no VertiPaq compression | Remove — store externally if needed             |

## LESSON 4

**Table.Buffer — Strategic Caching in M****After completing this lesson, you will be able to:**

- Explain the double-evaluation problem and when Table.Buffer prevents it.
- Identify which queries benefit from buffering and which do not.
- Apply List.Buffer for list-level caching.
- Measure the refresh time impact of Table.Buffer with Performance Analyzer.

**The Double-Evaluation Problem**

M uses lazy evaluation — an expression is only evaluated when its value is needed. When a query result is referenced in two places in the same `let...in` expression, M may evaluate the full source query twice — once for each reference. For a slow query (API call, complex join, large CSV), this doubles the refresh time for no additional benefit. `Table.Buffer` forces the query to evaluate once and holds the result in memory; subsequent references use the cached copy.

```
// ■ PROBLEM: DimCustomer evaluated twice – once for JoinA, once for JoinB
let
DimCustomer = load_customers(), // slow query – runs TWICE
JoinA = Table.Join(FactSales, "CustID", DimCustomer, "ID", JoinKind.LeftOuter),
JoinB = Table.Join(FactReturns, "CustID", DimCustomer, "ID", JoinKind.LeftOuter)
in
Table.Combine({JoinA, JoinB})

// ✓ FIX: Buffer DimCustomer – evaluated ONCE, referenced twice from RAM
let
DimCustomer = Table.Buffer( load_customers() ), // evaluated once ✓
JoinA = Table.Join(FactSales, "CustID", DimCustomer, "ID", JoinKind.LeftOuter),
JoinB = Table.Join(FactReturns, "CustID", DimCustomer, "ID", JoinKind.LeftOuter)
in
Table.Combine({JoinA, JoinB})
```

| Syntax    | Table.Buffer( table as table, optional options as record ) as table                                                                                                                          |                                                      |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| Parameter | Type                                                                                                                                                                                         | Description                                          |
| table     | table                                                                                                                                                                                        | The table to force-evaluate and cache in memory.     |
| options   | record                                                                                                                                                                                       | Optional. Currently unused; reserved for future use. |
| Returns   | Forces full evaluation of table immediately and holds the result in memory. All subsequent references to the returned value use the cached data without re-evaluating the source expression. |                                                      |
| Example   | Table.Buffer( Web.Contents_Result_Table )                                                                                                                                                    |                                                      |
| Result    | Fully evaluated table held in RAM                                                                                                                                                            |                                                      |

**NOTE**

Table.Buffer consumes RAM proportional to the table size. Do NOT buffer every table — only buffer tables referenced more than once. Buffering a large table referenced only once wastes RAM and adds evaluation overhead.

|                  |                                                                                                                                                                       |                                                 |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <b>Syntax</b>    | <code>List.Buffer( list as list ) as list</code>                                                                                                                      |                                                 |
| <b>Parameter</b> | <b>Type</b>                                                                                                                                                           | <b>Description</b>                              |
| list             | list                                                                                                                                                                  | The list to force-evaluate and cache in memory. |
| <b>Returns</b>   | Forces full evaluation of list and holds the result in memory. Use when a list is used repeatedly inside a List.Generate loop or passed to multiple List.* functions. |                                                 |
| <b>Example</b>   | <code>List.Buffer( List.Distinct( FactSales[ProductKey] ) )</code>                                                                                                    |                                                 |
| <b>Result</b>    | Distinct product key list held in RAM                                                                                                                                 |                                                 |

**NOTE**

List.Buffer is especially valuable inside List.Generate — if the condition or next function re-evaluates the same list on every iteration, buffering it prevents repeated source queries.

### When to Buffer — Decision Guide

| Buffer — Table.Buffer Recommended            | Do NOT Buffer — Unnecessary Overhead              |
|----------------------------------------------|---------------------------------------------------|
| Table referenced 2+ times in same let...in   | Table referenced exactly once                     |
| Lookup table used in multiple joins          | Source table at start of a pipeline (not re-used) |
| Expensive API/web query used in two branches | Buffering adds overhead — only one reference      |
| List used repeatedly in List.Generate loop   | Large table: buffer consumes too much RAM         |
| Config table read in multiple function calls | Query is already fast (< 1 second)                |

## LESSON 5

## Refresh Architecture and Scheduling

**After completing this lesson, you will be able to:**

- Configure Incremental Refresh with correct M query structure.
- Understand Hybrid Tables and the real-time DirectQuery partition.
- Choose between full refresh and incremental refresh for different table types.
- Apply parallel refresh patterns using dataset partitioning.

**Full Refresh vs Incremental Refresh**

Full refresh re-loads the entire table on every scheduled refresh. For a fact table with 3 years of historical data, this means reading, transferring, compressing, and storing all rows every night — even if only today's transactions changed. Incremental Refresh divides the table into time-based partitions and refreshes only recent partitions.

| Full Refresh                                | Incremental Refresh                                   |
|---------------------------------------------|-------------------------------------------------------|
| Reads all rows from source on every refresh | Reads only recent rows (configurable window)          |
| Transfer: full table every time             | Transfer: only new/changed rows                       |
| Refresh time: proportional to table size    | Refresh time: proportional to new data only           |
| No special M query structure required       | Requires RangeStart/RangeEnd parameters + fold        |
| Works on all Power BI capacities            | Available on Power BI Pro, Premium, and PPU           |
| History always perfectly accurate           | Historical partitions locked — prevent source changes |

**Incremental Refresh M Query Requirements**

```
// STEP 1: Create two Power BI parameters (exact names required by the Service)
// Name: RangeStart Type: Date/Time Suggested Value: e.g. 1/1/2024 0:00:00
// Name: RangeEnd Type: Date/Time Suggested Value: e.g. 1/1/2025 0:00:00

// STEP 2: M query – RangeStart and RangeEnd MUST fold to the source
// The date filter MUST be the first filtering step and MUST use these parameters directly

let
Source = Sql.Database("server", "SalesDB"),
Orders = Source[Schema="dbo", Item="FactOrders"][Data],

// ← THIS FILTER MUST FOLD (right-click → View Native Query must be active)
Filtered = Table.SelectRows(Orders,
each [OrderDate] >= RangeStart and [OrderDate] < RangeEnd),
// ↑ Parameters used directly – no M function wrapping

Typed = Table.TransformColumnTypes(Filterd,
{{"OrderDate", type date},
{"Revenue", type number},
{"CustomerID", type text}})
in
Typed

// STEP 3: In Power BI Desktop – Table > Incremental Refresh settings
```

```
// Store rows in the last: 3 years (historical partitions – not refreshed)
// Refresh rows in the last: 10 days (rolling refresh window)
// Optionally: Detect data changes using a Last Modified column
```

**WARNING**

Incremental Refresh silently falls back to full refresh if the RangeStart/RangeEnd filter does not fold to the source. Always verify by right-clicking the date filter step in Power Query Editor — View Native Query must be clickable and must show a WHERE clause using the parameter values. If it is greyed out, fix the fold break before publishing the dataset.

## Hybrid Tables — The Real-Time Partition

A Hybrid Table combines Incremental Refresh partitions (Import mode, fast DAX) with a special DirectQuery partition for the most recent data (e.g. today). Historical partitions are refreshed incrementally; the DirectQuery partition always queries the source in real time. This provides the speed of VertiPaq for historical analysis and the freshness of DirectQuery for current-day data.

```
// Hybrid Table requirements:
// 1. Same M query structure as Incremental Refresh (RangeStart / RangeEnd)
// 2. Enabled via: Table > Incremental Refresh > Get the latest data in real time with DirectQuery
// 3. Requires Power BI Premium Per User (PPU), Premium, or Fabric capacity
// 4. The DirectQuery partition automatically covers the period after the last Import partition

// How partitions are structured (example: 3 years history, 10 days incremental, real-time DQ):
// Partition 1: 2022-01-01 → 2022-12-31 (Import, historical, never refreshed)
// Partition 2: 2023-01-01 → 2023-12-31 (Import, historical, never refreshed)
// Partition 3: 2024-01-01 → 2024-12-22 (Import, incremental – refreshed daily)
// Partition 4: 2024-12-23 → NOW (DirectQuery – always live from source)

// DAX queries automatically combine partition results transparently to the report user
```

## LESSON 6

## Large Models, Composite Models, and DirectQuery

**After completing this lesson, you will be able to:**

- Enable Large Model storage format for datasets exceeding 1 GB.
- Design a Composite Model combining Import and DirectQuery tables.
- Configure Dual storage mode for dimension tables.
- Build an aggregation table in M to accelerate DirectQuery visuals.

### Large Model Storage Format

By default, Power BI datasets are limited to 1 GB in Power BI Service. Large Model Storage Format removes this limit, allowing datasets to grow to hundreds of gigabytes (subject to Premium capacity). It uses a different on-disk format (ABFS) that is streamed from Azure storage rather than loaded entirely into RAM on query.

```
// Large Model is enabled in Power BI Service, not in M:
// Dataset Settings → Large dataset storage format → On

// M best practices for large models:
// 1. Ensure all fact tables use Incremental Refresh – loading a 100M row table
// on every refresh is impractical regardless of storage format

// 2. Apply aggressive column pruning in M – every column removed = less RAM needed
// Remove ALL columns not used by a DAX measure or a slicer/filter

// 3. Replace DateTime with Date everywhere
// DateTime on a 100M row table = potentially 800 MB for one column alone

// 4. Use integer surrogate keys on all relationships
// Integer key compression = near-zero storage; text key = significant storage

// 5. Apply Table.TransformColumnTypes with the smallest appropriate integer type:
Table.TransformColumnTypes(Source, {
{"ProductKey", Int64.Type}, // or Int32 if < 2 billion values
{"Quantity", Int64.Type},
{"Revenue", type number}, // Currency precision
{"IsActive", type logical} // logical = 1 bit per row in VertiPaq
})
```

### Composite Models and Dual Storage Mode

A Composite Model mixes Import and DirectQuery tables in the same dataset. The fact table can be DirectQuery (always live) while dimension tables are Import (fast VertiPaq lookups). Dual storage mode for dimensions means VertiPaq stores a copy for Import queries AND the dimension can serve as a pass-through for DirectQuery queries — giving optimal performance for both modes.

```
// Composite Model configuration (Power BI Desktop):
// 1. Connect to SQL Server fact table → set to DirectQuery
// 2. Connect to Excel dimension tables → set to Import
// 3. Set dimension table storage mode to Dual:
// Right-click table in Model view → Properties → Storage mode → Dual

// In M, dimensions loaded as Import should be typed and compressed:
let
DimProduct = Excel.Workbook(File.Contents("Products.xlsx"))[[Name="Products"]][Data],
```

```

Promoted = Table.PromoteHeaders(DimProduct),
// Remove high-cardinality columns
Pruned = Table.SelectColumns(Promoted,
{"ProductKey", "ProductName", "Category", "SubCategory", "IsActive"}),
// Type with smallest appropriate types
Typed = Table.TransformColumnTypes(Pruned, {
{"ProductKey", Int64.Type},
{"ProductName", type text},
{"Category", type text},
{"SubCategory", type text},
{"IsActive", type logical}
})
in
Typed

// Dual storage: Power BI automatically uses Import data for Import-mode queries
// and DirectQuery path for DirectQuery queries – best of both worlds

```

## Aggregation Tables in M

```

// Aggregation tables pre-compute GROUP BY results and store them as Import
// DirectQuery visuals that match the aggregation granularity use the Import agg table
// instead of querying the large DirectQuery fact table – dramatically faster

// Build an aggregation table in M (runs at refresh, stored in VertiPaq):
let
// Source is the DirectQuery fact (via Sql.Database)
FactDQ = Sql.Database("server", "db"){[Schema="dbo", Item="FactSales"]}[Data],

// Filter to current incremental window (agg table is refreshed daily)
Filtered = Table.SelectRows(FactDQ,
each [SaleDate] >= RangeStart and [SaleDate] < RangeEnd),

// Aggregate at Month × Product × Region grain
Agg = Table.Group(Filtered,
{"YearMonth", "ProductKey", "RegionKey"},
{{"Revenue", each List.Sum([Revenue]), type number},
{"OrderCount", each Table.RowCount(_), Int64.Type},
{"Quantity", each List.Sum([Quantity]), Int64.Type}}),

Typed = Table.TransformColumnTypes(Agg,
{{"YearMonth", Int64.Type}, {"ProductKey", Int64.Type},
{"RegionKey", Int64.Type}, {"Revenue", type number},
{"OrderCount", Int64.Type}, {"Quantity", Int64.Type}})
in
Typed

// In Model view: manage aggregations on FactSales_Agg
// Map Revenue → Sum of Revenue in FactSales
// Map OrderCount → Count rows in FactSales
// Power BI routes aggregate queries to FactSales_Agg automatically

```

## LESSON 7

## Advanced M Performance Patterns

**After completing this lesson, you will be able to:**

- Implement lazy loading with function queries to defer expensive evaluation.
- Build a diagnostic watermark query for refresh monitoring.
- Apply column pruning programmatically from a config table.
- Load refresh-time configuration from SharePoint without breaking folding.

**Pattern 1: Lazy Loading with Function Queries**

```
// A query that returns a FUNCTION defers evaluation until the function is called
// Useful for: expensive queries needed only in certain branches, test vs prod switching

// Create a blank query named "GetSalesData" – its result is a function
let
GetSalesData = (startDate as date, endDate as date) as table =>
let
Source = Sql.Database("server", "SalesDB"),
Orders = Source[["Schema"="dbo", "Item"="FactOrders"]][Data],
Filtered = Table.SelectRows(Orders,
each [SaleDate] >= startDate and [SaleDate] < endDate),
Typed = Table.TransformColumnTypes(Filterd,
{{"SaleDate", type date}, {"Revenue", type number}})
in
Typed
in
GetSalesData

// Other queries call it on demand – evaluation happens only when invoked:
CurrentYearSales = GetSalesData(#date(2024,1,1), #date(2025,1,1))
PriorYearSales = GetSalesData(#date(2023,1,1), #date(2024,1,1))
// Two calls → two SQL queries, each folding correctly with their own date range
```

**Pattern 2: Diagnostic Watermark Query**

```
// A lightweight monitoring query that records refresh metadata
// Load this as a table in the model – it shows when data was last refreshed

let
// Capture refresh timestamp
RefreshTime = DateTime.LocalNow(),

// Get row counts from key tables (these are fast COUNT(*) queries that fold)
FactSource = Sql.Database("server", "SalesDB"),
FactTable = FactSource[["Schema"="dbo", "Item"="FactOrders"]][Data],
FactCount = Table.RowCount(FactTable), // folds to COUNT(*)

// Get max date in fact table (folds to MAX(OrderDate))
MaxDate = List.Max(FactTable[OrderDate]),

// Assemble diagnostic record
Diagnostic = #table(
{"RefreshTimestamp", "FactRowCount", "MaxOrderDate", "RefreshStatus"},
{{RefreshTime, FactCount, MaxDate, "Success"}}
),
```

```
Typed = Table.TransformColumnTypes(Diagnostic,
{"RefreshTimestamp", type datetime},
{"FactRowCount", Int64.Type},
{"MaxOrderDate", type date},
{"RefreshStatus", type text}})
in
Typed

// Use in Power BI: a card visual showing "Data as of: [MaxOrderDate]"
// and an alert on RefreshStatus for monitoring dashboards
```

### Pattern 3: Column Pruning Pipeline from Config

```
// Keep only the columns listed in a SharePoint config list
// Adding/removing columns from the model requires only a SharePoint list update – no M edits

let
// Load approved column list from SharePoint
SPSite = SharePoint.Tables("https://contoso.sharepoint.com/sites/BI",
[ApiVersion=15]),
ColConfig = SPSite[Title="ModelColumns"][Content],
ActiveCols = Table.SelectRows(ColConfig, each [Active] = true),
KeepCols = ActiveCols[ColumnName], // list of approved column names

// Apply to the fact table
Source = Sql.Database("server","SalesDB"),
Fact = Source[Schema="dbo",Item="FactOrders"][Data],

// Select only approved columns – MissingField.Ignore handles columns not in source
Pruned = Table.SelectColumns(Fact, KeepCols, MissingField.Ignore),
Typed = Table.TransformColumnTypes(Pruned,
List.Transform(KeepCols, (col) => {col, type text})) // default type
in
Typed

// Benefit: no M code change needed when business requests a new column or column removal
// Governance: the SharePoint list is the single source of truth for the model schema
```

## DEMONSTRATION 1

## Diagnosing Model Bloat with DAX Studio

**Scenario:** A Power BI PBIX file has grown to 1.2 GB. Report pages are slow to load. The task is to use DAX Studio's VertiPaq Analyzer to identify the largest columns, diagnose cardinality problems, and implement M-level fixes to reduce the model to under 200 MB.

### Demonstration Steps

|   |                                                                                                                                                                                                                                                                                              |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Open DAX Studio. Connect to the Power BI Desktop model. Navigate to Advanced → View Metrics → VertiPaq Analyzer. Show the Columns tab sorted by Total Size descending.                                                                                                                       |
| 2 | Identify the top 5 largest columns: FactSales[CreatedAt] (DateTime, 287 MB), FactSales[OrderNotes] (free text, 194 MB), FactSales[TransactionGUID] (unique string, 312 MB), DimCustomer[EmailAddress] (near-unique text, 43 MB), FactSales[RawJSON] (JSON blob, 178 MB).                     |
| 3 | Fix 1: Replace FactSales[CreatedAt] (datetime) with FactSales[SaleDate] (date). Show the M change: <code>Table.AddColumn(..., "SaleDate", each Date.From([CreatedAt]), type date)</code> followed by <code>Table.RemoveColumns(..., {"CreatedAt"})</code> . Refresh and show new size: 4 MB. |
| 4 | Fix 2: Remove FactSales[TransactionGUID] — it is a surrogate used only for deduplication in ETL. After deduplication is confirmed, remove the column entirely in M. Saves 312 MB.                                                                                                            |
| 5 | Fix 3: Remove FactSales[OrderNotes] and FactSales[RawJSON] — these are not used in any DAX measure or visual filter. Remove in M. Saves 372 MB.                                                                                                                                              |
| 6 | Fix 4: Replace DimCustomer[EmailAddress] with a hashed or masked version for GDPR compliance: <code>Table.AddColumn(..., "EmailDomain", each Text.AfterDelimiter([Email], "@"), type text)</code> . Reduces cardinality from 800K to 4K.                                                     |
| 7 | Refresh and re-run VertiPaq Analyzer. Model size: 1,214 MB → 167 MB. Report load time: 18 seconds → 2.1 seconds.                                                                                                                                                                             |

**RESULT**

Model size reduced from 1.2 GB to 167 MB — an 86% reduction — through four M-level changes: datetime → date split, removal of three unused columns, and text cardinality reduction on email address. Report load time improved from 18 to 2.1 seconds.

## DEMONSTRATION 2

**Building a Star Schema from a Flat File**

**Scenario:** A client has a single 8-million-row flat CSV file containing all sales transactions with product, customer, and region attributes denormalised into every row. Build a proper star schema — FactSales, DimProduct, DimCustomer, DimRegion, and DimDate — entirely in M with integer surrogate keys.

**Demonstration Steps**

|   |                                                                                                                                                                                                                                           |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Load the flat CSV. Show the raw structure: 8M rows, 42 columns, many text attributes repeated on every row. Show VertiPaq Analyzer: 1.8 GB PBIX file.                                                                                     |
| 2 | Create DimProduct: Table.SelectColumns → Table.Distinct on ProductCode → Table.Sort → Table.AddIndexColumn("ProductKey",1,1,Int64.Type).                                                                                                  |
| 3 | Create DimCustomer: same pattern on CustomerID → assigns CustomerKey integer.                                                                                                                                                             |
| 4 | Create DimRegion: Table.SelectColumns({"RegionCode","RegionName","Country","Continent"}) → Table.Distinct → AddIndexColumn("RegionKey").                                                                                                  |
| 5 | Create DimDate using the pure M generator from Lesson 2. Show it produces 4,749 rows (13 years). Add DateKey integer column.                                                                                                              |
| 6 | Build FactSales: start from the flat file, join each dimension to add integer surrogate keys using Table.NestedJoin + Table.ExpandTableColumn. Then Table.RemoveColumns to drop all text attribute columns — keep only keys and measures. |
| 7 | Define relationships in Model view: FactSales[ProductKey] → DimProduct[ProductKey], etc. Refresh. Show VertiPaq Analyzer: 1.8 GB → 148 MB. Build a sales-by-category measure in DAX — runs in 0.3 seconds vs 12 seconds on the flat file. |

**RESULT**

Star schema with integer surrogate keys reduces model from 1.8 GB to 148 MB. DAX query performance improves by 40x on aggregate measures. Cross-filter between DimProduct and DimCustomer now works correctly via the fact table.

## LAB 10

## Performance Optimisation and Advanced Architecture

In this lab you audit a bloated Power BI model, apply M-level optimisations, build a star schema from a flat source, configure Incremental Refresh, and implement a Composite Model with an aggregation table.

## Lab Scenario

The Contoso BI team has inherited a 900 MB PBIX file built from a flat denormalised SQL view. Reports are slow, refresh takes 2.5 hours, and the file is approaching the 1 GB Power BI Service limit. Your objective: reduce the model to under 150 MB, cut refresh time to under 20 minutes, and implement incremental refresh.

## NOTE

Use DAX Studio (free) for all VertiPaq analysis. Use AdventureWorksDW or the provided Contoso flat file for source data. Complete exercises in order.

## Exercise 1: VertiPaq Audit and Cardinality Reduction (Estimated time: 25 minutes)

- Connect DAX Studio to your Power BI Desktop model. Open VertiPaq Analyzer. Sort the Columns tab by Total Size. Identify the five largest columns.
- For each large column, record: cardinality, data type, and whether it is used in any DAX measure, relationship, or visual filter in the current report.
- Remove all unused columns in M using `Table.RemoveColumns` or `Table.SelectColumns`. Apply `Table.RemoveColumns` for each column not used by any measure or slicer.
- Find any `DateTime` columns. Replace each with a `Date` column using `Table.AddColumn(..., each Date.From([DateTimeCol]), type date)` followed by `Table.RemoveColumns` to drop the original.
- Find any high-cardinality text ID columns (GUIDs, transaction references). Replace with integer surrogate keys or remove after verifying they are not used in relationships.
- Re-run VertiPaq Analyzer. Record before and after model size. Target: at least 40% size reduction from column changes alone.
- STRETCH: Apply `Number.Round` to all revenue/price columns to reduce decimal precision from 6dp to 2dp. Measure the cardinality change in VertiPaq Analyzer.

## RESULT

Model size reduced by at least 40% from column pruning and `DateTime` → `Date` conversion. VertiPaq Analyzer confirms reduced cardinality on all modified columns.

## Exercise 2: Star Schema Construction in M (Estimated time: 30 minutes)

- Starting from the denormalised flat source, identify all dimension attributes (product, customer, region, date). List them in a design document.
- Build `DimProduct` in M: `SelectColumns` → `Distinct on ProductCode` → `Sort` → `AddIndexColumn`. Verify: one row per product, integer `ProductKey` starting at 1.
- Build `DimCustomer` in M: same pattern on `CustomerID`. Remove `EmailAddress` (GDPR) — replace with `EmailDomain` using `Text.AfterDelimiter`.
- Build `DimRegion` in M: extract `Region`, `Country`, `Continent`. Dedup on `RegionCode`.
- Build `DimDate` using the pure M generator from Lesson 2. Set `StartDate` to the earliest date in your fact data. Add `DateKey` integer column.

- Rebuild FactSales: join all four dimensions, add their integer keys, remove all text attribute columns. The fact table should have only integer keys and numeric measures.
- Define all relationships in Model view. Verify cross-filter works: selecting a Category in DimProduct should filter FactSales and DimCustomer visuals.
- STRETCH: Add a role-playing dimension for DimDate. Create two references: DimOrderDate and DimShipDate. Connect both to FactSales on their respective date keys.

## RESULT

A fully normalised star schema with five tables: FactSales (integer keys + measures), DimProduct, DimCustomer, DimRegion, DimDate. Model size at least 60% smaller than the original flat table. All cross-filters work correctly.

### Exercise 3: Incremental Refresh and Table.Buffer (Estimated time: 25 minutes)

- Create RangeStart and RangeEnd parameters of type Date/Time in Power BI Desktop.
- Modify the FactSales M query to use these parameters in a Table.SelectRows step. Verify the filter folds: right-click the step → View Native Query must be active.
- Configure Incremental Refresh: store 2 years of history, refresh the last 10 days. Optionally enable 'Detect data changes' on the LastModified column.
- Identify the DimCustomer query — it is referenced in two join steps (FactSales join and a second validation join). Add Table.Buffer(DimCustomer) and measure the refresh time difference.
- Apply List.Buffer to the distinct ProductKey list used in a List.Generate loop. Verify the list is evaluated only once per loop execution.
- STRETCH: Configure a Hybrid Table. Enable 'Get latest data in real time with DirectQuery' in the Incremental Refresh dialog. Verify the DirectQuery partition in Table Properties.

## RESULT

Incremental Refresh correctly configured and verified as folding. Table.Buffer reduces refresh time when DimCustomer is referenced twice. Hybrid Table shows a DirectQuery partition in Table Properties.

### Exercise 4: Composite Model and Aggregation Table (Estimated time: 25 minutes)

- Connect the largest fact table (FactSales) as DirectQuery from SQL Server. Connect DimProduct, DimCustomer, DimDate as Import from the existing model.
- Set DimProduct, DimCustomer, and DimDate storage mode to Dual in Model view.
- Build FactSales\_Agg in M: group FactSales (via Sql.Database, foldable) by YearMonth × ProductKey × RegionKey, computing Revenue SUM and OrderCount COUNT.
- Set FactSales\_Agg storage mode to Import. In Model view: Manage Aggregations → map Revenue/OrderCount to FactSales.
- Build a bar chart: Revenue by ProductCategory using a DAX SUM measure. Enable Performance Analyzer. Compare query time: FactSales (DirectQuery) vs FactSales\_Agg (Import aggregation match).
- Add a detail filter that forces row-level detail (e.g. filter by a specific OrderID). Show that Power BI automatically uses DirectQuery for the detail query and Import agg for the summary — no manual switching required.
- STRETCH: Add a Diagnostic Watermark query (Pattern 3 from Lesson 7). Add a card visual to the report showing 'Data refreshed: [RefreshTimestamp]'.

## RESULT

Composite Model with DirectQuery fact and Import dimensions. Aggregation table serves summary visuals from VertiPaq. Performance Analyzer confirms aggregate queries use the agg table. Detail queries fall through to DirectQuery automatically.

## Module Review and Takeaways

---

### Review Questions

---

#### What is cardinality and why does it directly determine VertiPaq compression quality?

**Answer:** Cardinality is the number of distinct values in a column. VertiPaq compresses columns using dictionary encoding (maps each distinct value to a small integer) and run-length encoding (consecutive repeated values stored as value+count). Low cardinality means few distinct values — the dictionary is small and the integer codes repeat heavily, giving near-perfect compression. High cardinality (especially unique values) means the dictionary IS the data — no compression is possible. A column where every row is unique compresses at less than 5%.

---

#### Why is storing a DateTime column on a fact table an anti-pattern, and how do you fix it?

**Answer:** A datetime value with microsecond precision is near-unique on a large fact table — each transaction has a different timestamp. VertiPaq cannot compress a near-unique column: a 10-million-row fact table with one DateTime column can be 200–300 MB for that column alone. The fix is to split it: add a Date column (`Date.From([datetime])`) which has at most ~365 distinct values per year, then remove the original datetime column. Date compresses to near-zero.

---

#### What does Table.Buffer do and when should you use it?

**Answer:** Table.Buffer forces a query expression to be fully evaluated once and held in memory. All subsequent references to the buffered result use the in-memory copy without re-evaluating the source. Use it when a table is referenced in two or more places in the same let...in expression — for example, a lookup dimension used in two join steps. Without buffering, M may evaluate the source query separately for each reference, doubling refresh time. Do not buffer tables referenced only once — it adds overhead without benefit.

---

#### Why is a star schema more performant in Power BI than a single flat denormalised table?

**Answer:** In a flat table, dimension attributes (product name, category, customer name, region) are repeated on every fact row. VertiPaq stores millions of copies of the same strings, consuming enormous RAM. In a star schema, the fact table stores only integer surrogate keys (low cardinality, near-zero storage), and dimension tables store each attribute value once. Additionally, star schemas enable DAX relationships, cross-filtering between dimensions, and role-playing dimensions — none of which are possible in a flat table.

---

#### What is the key M query requirement for Incremental Refresh to work correctly?

**Answer:** The RangeStart and RangeEnd parameters must fold directly to the data source — they must appear in a Table.SelectRows condition without any M function wrapping. The condition must be: each `[DateColumn] >= RangeStart` and `[DateColumn] < RangeEnd`. Right-clicking this step must show View Native Query (active, not greyed out). If the fold chain is broken before this filter, Power BI silently falls back to full refresh, defeating incremental refresh.

---

#### What is a Composite Model and what problem does it solve?

**Answer:** A Composite Model mixes Import and DirectQuery tables in the same Power BI dataset. It solves the trade-off between Import mode (fast DAX, data staleness) and DirectQuery mode (always live, slower queries). The typical pattern: large fact tables are DirectQuery (live data), dimension tables are Import (fast VertiPaq lookups), and dimension storage mode is set to Dual so they serve both Import and DirectQuery queries optimally.

---

**What is an aggregation table and how does Power BI use it?**

**Answer:** An aggregation table is a pre-computed GROUP BY summary stored in Import mode, mapped to a DirectQuery fact table via 'Manage Aggregations'. When a DAX query requests a summary at the aggregation grain (e.g. Revenue by Month × Product), Power BI automatically routes it to the Import agg table — returning VertiPaq speed. When a query requests row-level detail not covered by the aggregation, Power BI automatically falls through to the DirectQuery fact table. No DAX or report changes needed.

---

**How does DimDate built in M compare to an auto-generated date table in Power BI?**

**Answer:** Power BI auto-generates a hidden date table when 'Auto date/time' is enabled — it creates one table per date column, consuming significant model RAM. A single M-built DimDate is more efficient: one table, explicitly designed with all required attributes, shared across all date relationships. It also supports custom fiscal calendars, holiday flags, and any attribute needed by the business — impossible with the auto-generated table.

---

## Best Practices

---

- Run DAX Studio VertiPaq Analyzer on every model before publishing. Sort by Total Size and investigate any column consuming more than 5% of total model RAM.
- Never load DateTime columns on fact tables. Always split to Date and optionally HourOfDay. DateTime on a 10M-row table is typically 150–300 MB; Date is 2–5 MB.
- Remove all columns from M that are not used by a DAX measure, a relationship, a visual field, or a slicer. Unused columns consume RAM but return no value.
- Use integer surrogate keys on all fact-to-dimension relationships. Integer key columns compress to near-zero in VertiPaq. Text key columns (GUIDs, codes) have high cardinality.
- Build a single shared DimDate in M rather than enabling Power BI's auto date/time. Auto date/time creates one hidden date table per date column — massive RAM waste.
- Apply Table.Buffer only to tables referenced more than once in the same query. Buffering unnecessarily wastes RAM and adds evaluation overhead.
- For Incremental Refresh: verify the RangeStart/RangeEnd filter folds by right-clicking the date filter step — View Native Query must be active. Test before publishing.
- Disable 'Auto date/time' in Power BI Desktop Options for every model. It creates hidden date tables for every date column — redundant with a proper DimDate.
- For Composite Models: set all dimension tables to Dual storage mode. Dual allows dimensions to serve both Import queries (fast) and DirectQuery queries (pass-through to source when the fact is DirectQuery).
- Build aggregation tables for DirectQuery fact tables with over 1 million rows. Map aggregations via Manage Aggregations. Summary visuals run at Import speed while detail queries still have live data access via DirectQuery fall-through.

## Common Issues and Solutions

---

### Model size grows significantly after adding a few new columns

**Cause:** The new columns are DateTime type, free-text, or GUIDs — all high cardinality. Even one DateTime column on a large fact table can add 200 MB to the model.

**Solution:** Check VertiPaq Analyzer immediately after adding columns. Replace DateTime with Date, replace GUIDs with integer surrogate keys, remove free-text columns that are not analytically necessary.

### Incremental Refresh refreshes the full table despite being configured correctly

**Cause:** The RangeStart/RangeEnd date filter does not fold to the source. Power BI silently falls back to full refresh when folding fails.

**Solution:** Right-click the date filter step in Power Query Editor — View Native Query must be active (not greyed out). If greyed out, find and fix the fold break above the date filter step.

### Table.Buffer causes out-of-memory errors during refresh

**Cause:** A very large table is being buffered when it is only referenced once, or multiple large tables are buffered simultaneously, exhausting available RAM.

**Solution:** Only buffer tables referenced more than once. Remove unnecessary Table.Buffer calls. For Premium capacities, increase the Memory limit in Admin Portal settings.

### DimDate is missing dates needed for new reporting periods

**Cause:** The EndDate in the DimDate M query was hardcoded to a date that has now passed.

**Solution:** Replace the hardcoded EndDate with a dynamic expression:

`Date.AddYears(Date.From(DateTime.LocalNow()), 2)` to always generate 2 years ahead. Or set EndDate to `#date(2035,12,31)` and accept that the table grows by 365 rows/year.

---

### **Aggregation table does not get used — queries still hit DirectQuery**

**Cause:** The query grain does not match the aggregation table grain exactly, or the aggregation mapping is incorrect (wrong column or wrong function).

**Solution:** In Manage Aggregations, verify each measure is mapped to the correct column and aggregation function. Test with a simple visual exactly matching the agg grain. Enable Performance Analyzer to confirm whether queries hit Import or DirectQuery.

---

## Key Functions and Concepts — Module 10 Quick Reference

| Function / Concept                          | Category      | Performance Impact / Use                                   |
|---------------------------------------------|---------------|------------------------------------------------------------|
| <code>Table.Buffer(t)</code>                | Caching       | Force-evaluate table; cache in RAM; prevent double-eval    |
| <code>List.Buffer(l)</code>                 | Caching       | Force-evaluate list; cache in RAM for repeated use         |
| <code>Table.AddIndexColumn(t, ...)</code>   | Schema        | Integer surrogate key — use <code>Int64.Type</code>        |
| <code>Date.From([datetime])</code>          | Cardinality   | Extract date from datetime — reduces cardinality massively |
| <code>Table.SelectColumns(t, cols)</code>   | Pruning       | Keep only named cols — prune unused columns in M           |
| <code>Table.RemoveColumns(t, cols)</code>   | Pruning       | Drop named cols — use when keeping most columns            |
| <code>List.Dates(start, count, step)</code> | DimDate       | Generate list of sequential dates for DimDate              |
| <code>Number.Round(n, dp)</code>            | Cardinality   | Round decimals to reduce distinct values                   |
| <code>Table.Distinct(t, cols)</code>        | Dimension     | Deduplicate to build dimension tables                      |
| <code>Value.NativeQuery(src, sql)</code>    | Performance   | Raw SQL passthrough — push complex SQL to source           |
| <code>RangeStart / RangeEnd</code>          | Incr. Refresh | Required parameter names for Incremental Refresh           |
| <code>Table.TransformColumnTypes</code>     | VertiPaq      | Always type all columns — untyped = poor compression       |
| <code>Table.NestedJoin + Expand</code>      | Star Schema   | Add surrogate keys to fact table from dimension            |
| <code>Text.AfterDelimiter</code>            | Cardinality   | Extract domain from email — reduces text cardinality       |
| <code>DateTime.LocalNow()</code>            | Watermark     | Current refresh timestamp for diagnostic query             |

## Module Summary

- VertiPaq is a column-store engine that compresses using dictionary + run-length encoding. Compression quality is determined by cardinality. Low-cardinality columns compress to near-zero; high-cardinality (especially unique) columns consume enormous RAM. Always measure with DAX Studio VertiPaq Analyzer before and after M changes.
- Star schema construction in M: extract dimension tables with `Table.Distinct`, assign integer surrogate keys with `Table.AddIndexColumn`, join keys back to the fact with `Table.NestedJoin`, then remove all text attribute columns from the fact. The result is a lean, fast model.
- The two most impactful cardinality reductions: (1) replace `DateTime` with `Date` using `Date.From()` — reduces cardinality from millions to hundreds; (2) remove unused high-cardinality columns (GUIDs, free text, audit columns) entirely in M.
- `Table.Buffer` prevents double-evaluation of queries referenced multiple times. Use strategically — only on tables referenced 2+ times. Do not buffer single-reference tables.
- Incremental Refresh requires the `RangeStart/RangeEnd` filter to fold. Verify with View Native Query. Hybrid Tables add a live `DirectQuery` partition for the most recent data while historical partitions remain fast Import.
- Composite Models combine Import dimensions (fast VertiPaq) with `DirectQuery` fact tables (live data). Dual storage on dimensions serves both modes. Aggregation tables route summary queries to Import automatically while detail queries fall through to `DirectQuery`.
- Five advanced M patterns: lazy loading with function queries, parallel requests, diagnostic watermark, config-driven column pruning from SharePoint, and refresh-time config injection.

### NOTE

Next Module — Module 11: Real-World Power BI Architecture. Covers enterprise deployment patterns, workspace architecture, dataflow vs dataset, gateway configuration for on-premises sources, row-level security implementation in M, and the complete end-to-end design of a production-grade Power BI platform.